

VISHVAKARMA.OS

Complete Software Summary Document

iPad-First Architectural Floor Plan Studio

React 18 · TypeScript 5 · Vite 5 · Three.js · Tailwind CSS

Lovable Cloud Backend · Real-time Collaboration

April 2026 · Release Candidate

Table of Contents

1. Executive Overview
2. Technology Stack & Architecture
3. Core Engine (FloorPlanEngine)
4. 2D Canvas Editor
5. 3D Viewer & Sculptor
6. Simulation Engines
7. UI System & Design Language
8. Cloud Backend & Persistence
9. Authentication & Collaboration
10. Pages & Routing
11. Component Inventory
12. Export Capabilities
13. Template System
14. Testing & Quality
15. Commercial Model
16. Known Limitations & Roadmap

1. Executive Overview

Vishvakarma.OS is a professional-grade, iPad-first architectural floor plan studio that combines 2D drafting, 3D visualization, MEP (Mechanical/Electrical/Plumbing) routing, Vastu Shastra analysis, CFD wind simulation, and real-time collaboration into a single browser-based application.

Named after Vishvakarma, the divine architect of Hindu mythology, the software blends cutting-edge web technologies with a culturally-rooted 'Sacred UX' design language featuring obsidian-and-gold aesthetics, Devanagari labels, and meditative interaction patterns.

Key Differentiators:

- Zero-install — runs entirely in the browser on iPad and desktop
- Five workspace modes: Draft, MEP, Interior, Landscape, Walk
- Real-time 2D→3D crossfade with PBR materials and volumetric lighting
- Integrated Vastu Shastra compass with harmony scoring
- WebGPU-accelerated CFD wind simulation (Vayu engine)
- A* pathfinding MEP routing (Tvashtar engine)
- CSG Boolean operations for wall cutouts (Brahma engine)
- Cloud sync with real-time multi-user collaboration
- Export to PNG, PDF, and DXF formats

2. Technology Stack & Architecture

- Frontend Framework:** React 18 with TypeScript 5, bundled by Vite 5
- Styling:** Tailwind CSS v3 with custom design tokens (HSL-based), shadcn/ui components
- 3D Rendering:** Three.js r170 via @react-three/fiber and @react-three/drei
- State Management:** Custom singleton engine with useSyncExternalStore + Zustand (uiStore)
- Backend:** Lovable Cloud (Supabase) — PostgreSQL, Auth, Realtime, Edge Functions, Storage
- Animation:** Framer Motion for UI transitions
- Routing:** React Router v6
- Testing:** Vitest (unit) + Playwright (E2E)

Architecture Pattern

The application follows a **singleton engine + reactive UI** pattern. The FloorPlanEngine class (src/core/engine.ts) holds all project state and exposes it via a version-bumping subscriber pattern. React components subscribe using useSyncExternalStore through the useEngine hook. The engine handles undo/redo, room detection, cost estimation, and persistence, while React components remain purely presentational.

Directory Structure

Directory	Purpose
src/core/	Engine, types, geometry, simulations, exporters, templates
src/components/floor-plan/	All editor UI components (40+ files)
src/components/ui/	shadcn/ui primitives (50+ components)
src/pages/	Route-level pages (Landing, Auth, Features, Pricing, etc.)
src/hooks/	Custom hooks (useAuth, useCollaboration, useMobile)
src/stores/	Zustand UI store
src/core/types/	Domain type definitions split by concern
remotion/	Video generation project for tutorial content

3. Core Engine (FloorPlanEngine)

The FloorPlanEngine (src/core/engine.ts, ~1,084 lines) is the brain of the application. It is a singleton class that manages all project data and editor state.

State Management

- **Project Data:** Walls, doors, windows, furniture, rooms, MEP symbols, plumbing, landscape elements, measurements, cost items, staircases, ceiling zones
- **Editor State:** Active tool, selected entity, zoom/pan, grid settings, layer visibility, workspace mode, zen mode, presentation lock
- **Undo/Redo:** Full state snapshots pushed onto undo/redo stacks with configurable depth
- **Persistence:** Auto-saves to localStorage on every state change; cloud sync via interval timer

Key Operations

Method	Description
addWall(start, end, controlPoint?, thickness?)	Creates wall with optional curve
addDoor(wallId, offset, swing?)	Places door on wall
addFurniture(type, x, y)	Drops furniture item
detectRooms()	Graph-based room detection from wall topology
deleteSelected()	Removes selected entity
undo() / redo()	State stack navigation
loadTemplate(data)	Applies floor plan template
toggleZenMode()	Hides all chrome for distraction-free drawing

4. 2D Canvas Editor

The 2D editor (CanvasView.tsx, ~1,005 lines) is an HTML5 Canvas-based drafting surface with a finite state machine (FSM) driving interactions.

Interaction States

- IDLE → waiting for user action
- DRAWING_WALL → click-to-place wall endpoints with smart snapping
- PLACING_DOOR / PLACING_WINDOW → click wall to position opening
- PLACING_FURNITURE → click canvas to drop furniture item
- SELECTING → click/drag to select and move entities
- PANNING → two-finger or middle-mouse pan
- MEASURING → click-to-click dimension tool

Rendering Pipeline

The renderer (renderer.ts + sub-renderers) draws all elements each frame using Canvas 2D API:

- Grid layer (dots or lines, configurable spacing)
- Walls with thickness, color, and material patterns
- Doors with swing arc visualization
- Windows with glass fill indication
- Furniture with type-specific icons
- Room labels with area calculations
- MEP symbols and plumbing runs
- Dimensions and measurements
- Smart guides and snap indicators
- Vastu compass overlay
- CFD wind particle animation
- Selection highlights and handles

Input Features

- Apple Pencil pressure sensitivity with PencilPro module
- Two-finger undo gesture, three-finger redo
- Pencil double-tap to switch tools
- Smart snapping to grid, endpoints, midpoints, and angles (15° increments)
- Constraint dimensioning — type exact lengths during wall drawing
- Long-press radial menu for quick sub-tool access

5. 3D Viewer & Sculptor

The 3D Viewer (Viewer3D.tsx, ~1,891 lines) provides a real-time 3D visualization of the floor plan using React Three Fiber (Three.js).

Features

- **Automatic Extrusion:** 2D walls become 3D meshes with configurable height
- **PBR Materials:** 24+ physically-based materials (marble, brick, wood, concrete, etc.) with procedural texture generation
- **Material Painting:** Click any surface to apply a material from the palette
- **Volumetric Lighting:** God rays, dust motes, ambient occlusion
- **Section Box:** Clip the scene to inspect interior spaces
- **First-Person Walker:** WASD navigation through the space
- **Foundry Mode:** Strips UI for cinematic presentations with vignette and film grain
- **World Outliner:** Scene tree panel with Devanagari labels
- **Multi-floor Support:** Add/manage multiple floor levels
- **Ceiling Zones:** Tray, coffered, and vaulted ceiling types
- **Landscape:** Terrain, garden paths, procedural trees

Srishti Gesture

A unique two-finger pull-up gesture that smoothly crossfades from the 2D canvas to the 3D viewer, with a progress indicator showing the transition.

6. Simulation Engines

Vayu (Wind CFD) — [vayuCFD.ts](#) / [vayuGPU.ts](#)

- Computational Fluid Dynamics for cross-ventilation analysis
- CPU solver with Navier-Stokes approximation
- WebGPU-accelerated solver for real-time performance
- Velocity field visualization with particle animation

Agni (Thermal) — [thermalEngine.ts](#)

- Heat transfer simulation between rooms
- R-value calculations based on wall materials
- Temperature differential modeling
- Thermal comfort scoring

Vastu (Harmony) — [vastu.ts](#)

- 8-direction Vastu Shastra analysis
- Room placement scoring against ideal zones
- Main entrance direction evaluation
- Kitchen and master bedroom optimization
- Overall harmony percentage with actionable tips

Tvashtar (MEP Routing) — [tvashtar.ts](#)

- A* pathfinding for pipe and wire routing
- Collision-aware path planning around walls
- Cost estimation for materials
- 3D routing visualization

Brahma (CSG Booleans) — [brahmaCSG.ts](#)

- Constructive Solid Geometry for wall cutouts
- Door and window aperture boolean subtraction
- Niche carving operations

Jala (Water Caustics) — [jalaCaustics.ts](#)

- Decorative water caustic effects for pools/features
- GPU-computed caustic patterns

7. UI System & Design Language

The 'Sacred UX' design language creates a premium, culturally-rooted experience:

Color Palette

Token	Value	Usage
Obsidian Background	hsl(0 0% 2-3%)	Deep black panels
Gold Accent	hsl(42 60% 56%)	Primary interactive color
Muted Gold	hsl(42 35% 38%)	Secondary gold for borders
Canvas Paper	#f6f1e7	Warm drafting paper texture
Chrome	hsl(0 0% 3%)	Top bar and tool rail

Signature UI Elements

- **Glassmorphism panels:** backdrop-blur(40px) with obsidian tint
- **Devanagari labels:** Laser-etched CSS glow effect on Sanskrit terms
- **SriYantra Loader:** Sacred geometry loading animation
- **Creation Ritual:** Animated startup sequence with breathing overlay
- **Lotus Menu:** Context menu with petal-shaped radial layout
- **Command Dial:** Rotary control for adjusting wall thickness, angles
- **Zen Mode:** Hides all UI chrome for distraction-free drafting
- **Prana Pulse:** Subtle breathing animation for meditation moments

Responsive Behavior

The entire UI is optimized for iPad (1024x1366) as the primary target, with full desktop support. Touch gestures include pinch-zoom, two-finger pan, three-finger swipe for Zen Mode toggle, and Apple Pencil pressure sensitivity.

8. Cloud Backend & Persistence

The application uses Lovable Cloud for all backend services:

Database Tables

Table	Columns
projects	id, owner_id, name, data (JSON), is_public, share_code, timestamps
profiles	user_id, display_name, firm_name, avatar_url, timestamps
collaborators	project_id, user_id, role, timestamps

Persistence Strategy

- **Local:** Auto-saves to localStorage on every engine state change
- **Cloud:** Debounced save every 5 seconds when authenticated
- **Realtime:** Subscribes to project changes for cross-device sync
- **Sharing:** Generate share codes that create public URLs
- **Security:** Row-Level Security (RLS) policies on all tables

9. Authentication & Collaboration

Auth System: Email/password authentication with email verification via Lovable Cloud Auth. Protected routes redirect unauthenticated users to /auth.

Real-time Collaboration: Multiple users can edit the same project simultaneously. Live cursor positions are broadcast via Realtime channels. Collaborator avatars appear in the top-right corner with color-coded indicators.

User Profiles: Display name, firm name, avatar URL stored in profiles table. Profile page at /profile for account management.

10. Pages & Routing

Route	Page	Description
/	Landing	Marketing page with hero video, feature highlights, testimonials, CTA
/features	Features	Two-tab layout: Video Guides (6 tutorials) and All Features grid
/pricing	Pricing	Three-tier pricing: Starter (Free), Studio (\$99), Enterprise (\$249)
/auth	Auth	Sign up / Sign in with email verification
/reset-password	Reset Password	Password recovery flow
/editor	Editor (Index)	Protected — main floor plan workspace
/projects	Projects	Protected — project dashboard with CRUD
/profile	Profile	Protected — user account settings
/*	404	Not found page

11. Component Inventory

The floor-plan directory contains 40+ specialized components:

CanvasView — Main 2D drafting canvas with FSM-driven interactions

Viewer3D — Full 3D scene with PBR materials and atmospheric effects

TopBar — Project name, mode switcher, undo/redo, export, settings

ToolRail — Contextual left sidebar with mode-specific tool palettes

RightDrawer — Unified inspector panel with Properties and Layers tabs

CommandPalette — Cmd+K fuzzy search for tools, furniture, actions

VastuPanel — Vastu Shastra harmony analysis display

KarmaTimeline — Version control branching with mandala visualization

CostEstimator — Real-time construction cost breakdown

PhaseTimeline — Construction phase planning

SchemaTabs — MEP/Acoustic/Thermal overlay toggles

PanchatattvaPanel — Five-element architectural balance analysis

TvashtarPanel — MEP routing configuration and results

AgniThermalPanel — Thermal simulation settings and heatmap

VayuJalaPanel — Wind + water simulation controls

AkashaCastPanel — Cloud rendering / casting controls

DrishtiAR — WebXR augmented reality overlay

FirstPersonWalker — WASD 3D walkthrough controls

CreationRitual — Startup animation sequence

SplashScreen — Brand loading screen

WelcomeOverlay — First-time user onboarding

CommandDial — Rotary property adjustment control

ExportModal — PNG/PDF/DXF export dialog

PresentationLock — Lock UI for client presentations

VakyaVoice — Voice command interface

SrishtiGesture — Two-finger 2D→3D pull gesture

TemplatePicker — Floor plan template selection

PencilHover — Apple Pencil hover preview indicator

SmartReticle — Snap point crosshair display

StatusPill — Tool state and snap mode indicator

HintBar — Contextual keyboard shortcut hints

12. Export Capabilities

- **PNG Export:** High-resolution rasterized floor plan with all layers
- **PDF Export:** Vector-quality PDF with room labels, dimensions, and legend (pdfExport.ts)
- **DXF Export:** AutoCAD-compatible DXF with walls as LINE/ARC entities, doors, windows, and dimensions on separate layers (dxfExport.ts)
- **Share Link:** Generate public URL with unique share code for view-only access

13. Template System

Three built-in templates provide instant starting points:

- **Studio Apartment:** Open-plan layout with kitchen, bathroom, and living area
- **2BHK Apartment:** Two-bedroom layout with living room, kitchen, and two bathrooms (default demo project)
- **3BHK Apartment:** Three-bedroom layout with separate dining, master suite, and utility areas

Templates are defined in src/core/templates.ts using helper functions that generate Wall, Door, and FurnitureItem arrays. New templates can be added by implementing the FloorTemplate interface.

14. Testing & Quality

Unit Tests (Vitest): 8 test suites covering geometry operations, CSG booleans, Vastu analysis, MEP routing, and morph transformations.

E2E Tests (Playwright): Core loop test validates Landing → Pricing → Auth → Editor initialization flow. Runs against Chrome and WebKit.

Type Safety: Strict TypeScript with no implicit any. Domain types split across 11 focused type files.

Code Splitting: Lazy-loaded routes with React.lazy and Suspense. Three.js and Viewer3D loaded on demand.

15. Commercial Model

Feature	Starter (Free)	Studio (\$99/mo)	Enterprise (\$249/mo)
Projects	2	Unlimited	Unlimited
Collaboration	—	Up to 5	Unlimited
Export Formats	PNG only	PNG, PDF, DXF	All + API access
3D Viewer	Basic	Full PBR + Foundry	Full + AR mode
Vastu Analysis	Basic score	Full breakdown	Full + custom rules
Cloud Storage	100 MB	10 GB	Unlimited
Support	Community	Priority email	Dedicated manager

16. Known Limitations & Roadmap

Current Limitations:

- Monolithic files (engine.ts, Viewer3D.tsx, CanvasView.tsx) need refactoring
- WebGPU CFD requires compatible browser — falls back to CPU solver
- No offline PWA support yet
- Template library limited to 3 presets
- No multi-story staircase connections in 3D

Roadmap:

- PWA offline mode with service worker caching
- AI-powered room layout suggestions
- Import from DXF/DWG files
- Material cost API integrations for real-time pricing
- WebXR spatial computing for Apple Vision Pro
- Plugin marketplace for custom tools and materials